Natürliche Sprachverarbeitung mit KI

Von Regeln über N-Gramme zu Transformern

Erik Voigt, Johannes Langer

INHALT

§1	Natürliche Sprachverarbeitung: Grundlagen	03
§2	Probabilistische Sprachmodelle	06
§3	Vektorrepräsentationen	10
§4	Neuronale Sprachmodelle	13
§5	Zusammenfassung	17
	Literatur	18
	Anhang: Herleitung des Skalierungsfaktors	18
	Über die Autoren	19

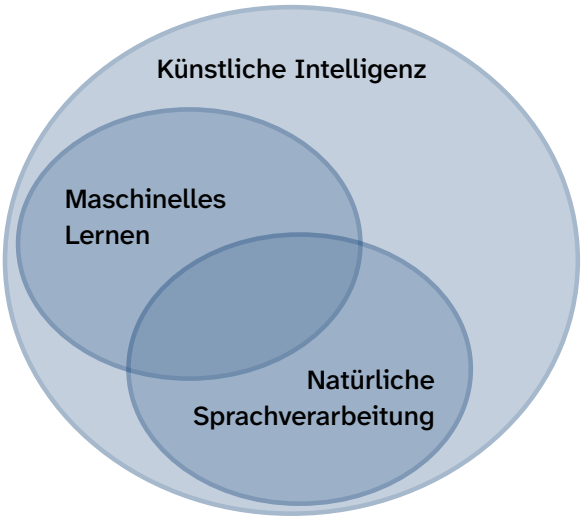
§1

Natürliche Sprachverarbeitung: Grundlagen

1.1 Was ist natürliche Sprachverarbeitung?

Die natürliche Sprachverarbeitung (*Natural Language Processing*, NLP) ist ein Teilbereich der Künstlichen Intelligenz. Sie fragt, wie Computer mit natürlicher, also menschlicher Sprache umgehen können, und wie sich relevante Informationen aus Texten extrahieren und verarbeiten lassen. Abb. 1 ordnet das Gebiet ein: Ein großer Teil der heutigen Sprachverarbeitung beruht auf maschinellem Lernen, aber nicht jede Methode der Sprachverarbeitung lernt aus Daten.

ABB. 1 Natürliche Sprachverarbeitung als Teilgebiet der Künstlichen Intelligenz, mit großer Schnittmenge zum maschinellen Lernen.



Die wichtigste Herausforderung ist dabei die *Ambiguität* von Sprache. Ein Satz kann syntaktisch mehrdeutig sein (welches Satzglied bezieht sich worauf?), ein Wort lexikalisch (eine *Bank* zum Sitzen oder für Geld), und vieles erschließt sich erst aus dem Kontext. Menschen lösen solche Mehrdeutigkeiten meist, ohne sie überhaupt zu bemerken; für ein Programm müssen sie explizit aufgelöst werden.

1.2 Ein historischer Ansatz: ELIZA

Als erster »Chatbot« gilt ELIZA (Weizenbaum, 1966), eine textbasierte Simulation eines Rogers'schen Psychotherapeuten. ELIZA ist ein einfaches Programm, das Regeln und Vorlagen nutzt, die von Expert:innen entworfen wurden. Die Eingabe wird in Schlüsselwörter zerlegt, die bestimmte Regeln auslösen, und die Antworten werden aus ebendiesen oder aus damit verknüpften Regeln erzeugt (Abb. 2). Wer ELIZA selbst ausprobieren möchte, findet eine Nachbildung unter masswerk.at/eliza.

ABB. 2 Ablaufdiagramm der Schlüsselworterkennung in ELIZA (Weizenbaum, 1966).

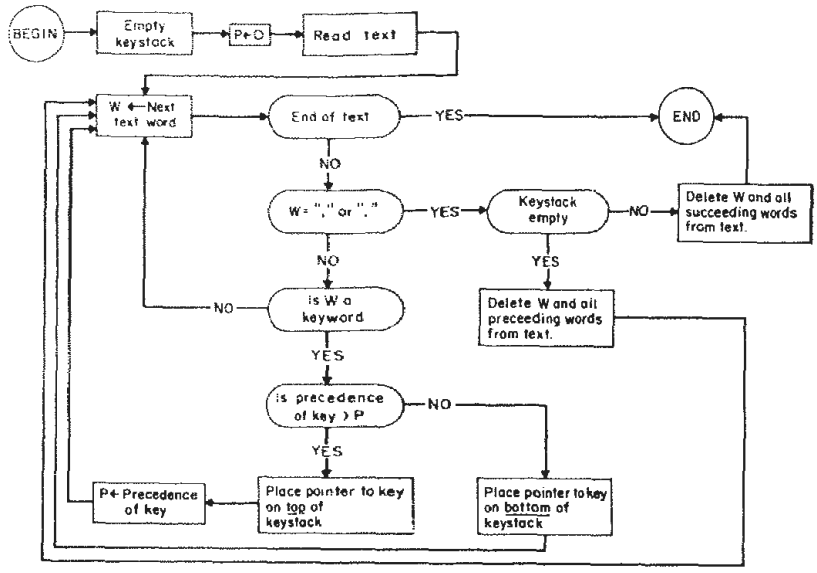
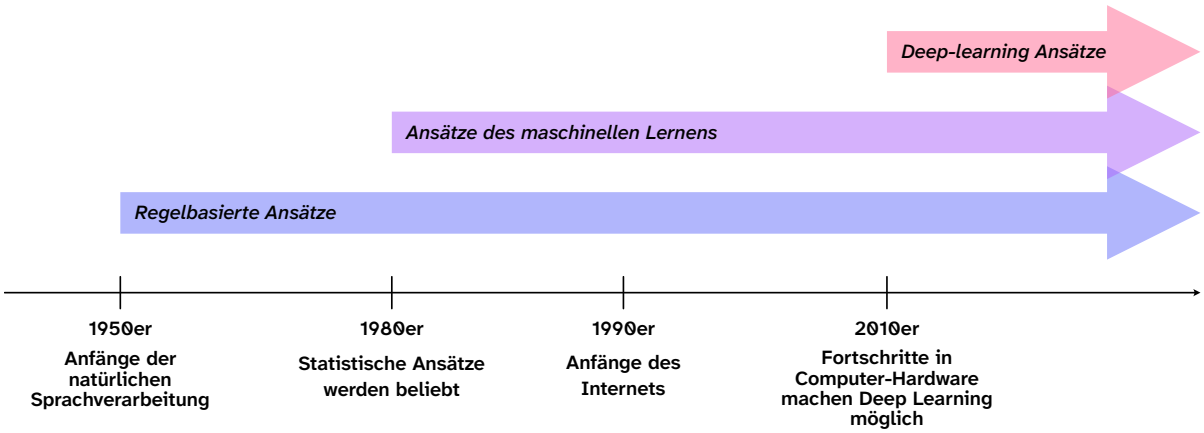


FIG. 2. Basic flow diagram of keyword detection

1.3 Von Regeln zu Statistik

Die Geschichte der Sprachverarbeitung lässt sich grob in regelbasierte und statistische Ansätze einteilen (Abb. 3). *Regelbasierte* Systeme brauchen keine Daten, sondern nutzen Expertenwissen, und sie können ihr Verhalten garantieren. *Statistische* Systeme benötigen dagegen Daten. Sie sind dort anwendbar, wo komplexe Regeln schwer zu formulieren oder zu finden sind, machen dafür aber manchmal Fehler (Kochmar, 2022).

ABB. 3 Zeitstrahl der Sprachverarbeitung, von regelbasierten Ansätzen über maschinelles Lernen zu Deep Learning (nach Kochmar, 2022).



1.4 Maschinelles Lernen

Was es heißt, dass ein Programm aus Daten »lernt«, hat Mitchell (1997) so definiert:

DEF 1.1 **Maschinelles Lernen**

Ein Computerprogramm **lernt** aus Erfahrung *E* im Bezug auf Aufgaben *T* und Leistungsmaß *P*, wenn dessen Leistung in Aufgaben *T*, gemessen durch *P*, mit Erfahrung *E* steigt (Mitchell, 1997).

Die drei Bestandteile der Definition sind bewusst allgemein gehalten:

Erfahrung Daten, repräsentiert als Mengen von Attributen (zum Beispiel demografische Daten und gesundheitliche Vorgeschichte), Bilder, Spielzüge oder Beobachtungen, etwa eine Folge von Ereignissen oder auch von Wörtern.

Aufgabe Zum Beispiel Klassifikation (hohes, mittleres oder geringes Krankheitsrisiko), Objekterkennung, Schach spielen oder das nächste Ereignis bzw. Wort vorhersagen.

Leistungsmaß Üblicherweise eine Form von *Genauigkeit*, also der Anteil korrekter Schätzungen auf *unbekannten* Daten, aber auch Interpretierbarkeit und *Anwendbarkeit* für Menschen.

Maschinelles Lernen hat großen praktischen Nutzen in sehr unterschiedlichen Bereichen. Beim *Data-Mining* enthalten große Datenbanken häufig wertvolle implizite Regelmäßigkeiten, die sich automatisch entdecken lassen, etwa in den Ergebnissen medizinischer Behandlungen oder in den Vorlieben von Kund:innen. In komplexen Domänen haben Menschen das nötige Wissen oft nicht oder können es nicht so formulieren, dass daraus ein effizienter Algorithmus entsteht; Gesichtserkennung auf Bildern ist ein typisches Beispiel. Andere Programme müssen sich dynamisch an veränderte Bedingungen anpassen, etwa um Fertigungsprozesse bei schwankenden Materialvorräten zu steuern. Und schließlich beruhen autonome Systeme wie Roboter oder selbstfahrende Autos wesentlich auf gelernten Komponenten.

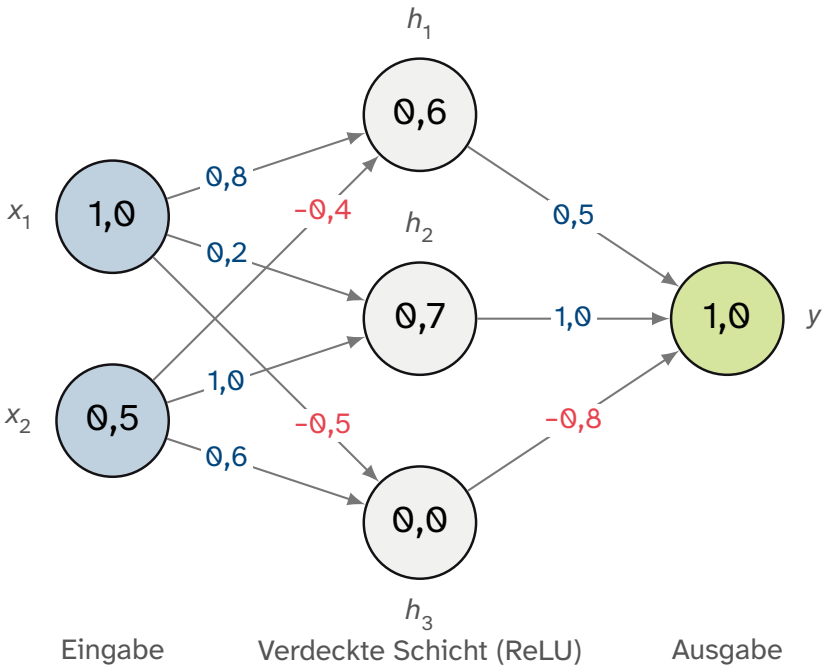
1.5 Neuronale Netze

Neuronale Netze sind nur einer von vielen Algorithmen für maschinelles Lernen, aber der für moderne Sprachmodelle entscheidende. Ein neuronales Netz ist eine differenzierbare Funktion mit (vielen) variablen, *lernbaren* Parametern, die anhand einer Fehlerfunktion *optimiert* werden; die Fehlerfunktion entspricht dem *Leistungsmaß* aus Def 1.1. Neuronale Netze bestehen häufig aus mehreren *Schichten*, wobei jede Schicht einer differenzierbaren Teilfunktion entspricht. Über die Kettenregel lässt sich so für jeden Parameter berechnen, wie er den Fehler beeinflusst.

Neuronale Netze arbeiten vollständig auf reellwertigen Ein- und Ausgaben. Viele Schichten lassen sich als Matrixmultiplikationen darstellen und können deshalb auf Grafikprozessoren sehr schnell berechnet werden.

In Abb. 4 berechnet jedes Neuron die gewichtete Summe seiner Eingaben und wendet darauf eine *Aktivierungsfunktion* an, hier $\mathrm{ReLU}(z) = \max(0, z)$. Für die Eingabe $x = (1{,}0;\ 0{,}5)$ ergibt sich:

ABB. 4 Ein kleines neuronales Netz mit zwei Eingaben, einer verdeckten Schicht und einer Ausgabe. In den Kreisen steht die Ausgabe des jeweiligen Neurons, an den Kanten das Gewicht.



$$h_1 = \max(0,\ 0{,}8 \cdot 1{,}0 - 0{,}4 \cdot 0{,}5) = 0{,}6$$
$$h_2 = \max(0,\ 0{,}2 \cdot 1{,}0 + 1{,}0 \cdot 0{,}5) = 0{,}7$$
$$h_3 = \max(0,\ -0{,}5 \cdot 1{,}0 + 0{,}6 \cdot 0{,}5) = 0$$
$$y = 0{,}5 \cdot 0{,}6 + 1{,}0 \cdot 0{,}7 - 0{,}8 \cdot 0 = 1{,}0$$

Die Gewichte sind die *lernbaren Parameter*: Beim Training werden sie so angepasst, dass y dem gewünschten Wert näher kommt.

§2

Probabilistische Sprachmodelle

2.1 Definition und Begriffe

Ein Sprachmodell beschreibt, wie wahrscheinlich eine Folge von Wörtern ist. Die übliche Formulierung zerlegt diese Wahrscheinlichkeit Wort für Wort (Bengio et al., 2003):

DEF 2.1 **Probabilistisches Sprachmodell**

Ein **statistisches** oder **probabilistisches Sprachmodell** kann durch die *bedingte Wahrscheinlichkeit* des nächsten Wortes, gegeben alle vorherigen Wörter, repräsentiert werden. Dann ist die Wahrscheinlichkeit einer Wortfolge

$$\hat{P}(w_1, \dots, w_T) = \prod_{i=1}^{T} \hat{P}(w_i \mid w_1, \dots, w_{i-1}). \tag{2.1}$$

Zwei Begriffe begleiten uns durch den Rest des Skripts:

DEF 2.2 **Token**

Ein **Token** ist die kleinste diskrete Einheit in der Sprachmodellierung. Token können Zeichen, Wortteile, Wörter oder sogar ganze Sätze sein.

DEF 2.3 **Vokabular**

Das **Vokabular** V ist die Menge der unterschiedlichen *Token* eines Sprachmodells. Die Größe des Vokabulars $|V|$ beschreibt den *Zustandsraum* des Modells. Token, die nicht im Vokabular enthalten sind, kennt das Modell nicht und kann sie nicht verarbeiten.

2.2 N-Gramm-Modelle

WICHTIG **Die Markov-Annahme**

In der Praxis ist es unmöglich, die Wahrscheinlichkeiten in Gl. (2.1) für beliebig lange Token-Folgen zu schätzen. Ein *N-Gramm-Sprachmodell* trifft deshalb die *Markov-Annahme* $(N-1)$-ter Ordnung: Die Wahrscheinlichkeit des Tokens i hängt nur von den vorherigen $N-1$ Token ab. N heißt auch die *Ordnung* des Modells.[1]

DEF 2.4 **N-Gramm-Sprachmodell**

Ein **N-Gramm-Sprachmodell** ist ein *probabilistisches Sprachmodell* unter der Markov-Annahme $(N-1)$-ter Ordnung:

$$\hat{P}(w_i \mid w_1, \dots, w_{i-1}) \approx \hat{P}(w_i \mid w_{i-(N-1)}, \dots, w_{i-1}) \tag{2.2}$$

1 Eine anschauliche Einführung in Markov-Ketten gibt das Video The Strange Math That Predicts (Almost) Anything von Veritasium (2025).

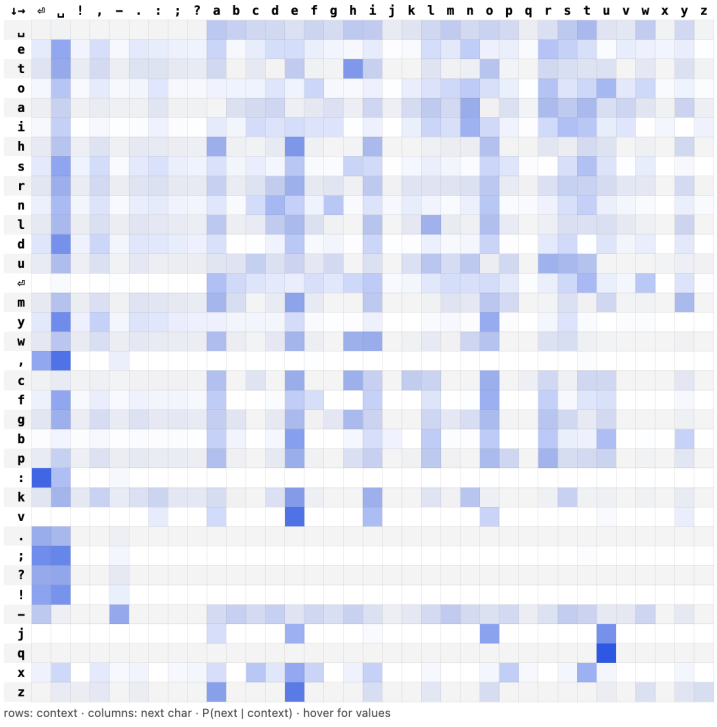
Im Workshop haben wir ein N-Gramm-Modell auf Zeichenebene selbst trainiert. Das zugehörige Notebook ist unter tinyurl.com/smw-ngram zu finden (Abb. 5).

ABB. 5 Notebook zu N-Gramm-Modellen: tinyurl.com/smw-ngram.

Training

Das Training eines N-Gramm-Modells veranschaulicht das empirische *Gesetz der großen Zahlen*: Wir schätzen die wahre, unsichtbare Wahrscheinlichkeit eines Folgetokens, indem wir einfach seine *relative Häufigkeit* in einem großen Textkorpus ermitteln. Für ein Bigramm-Modell auf Zeichenebene ergibt sich so eine Übergangsmatrix, in der jede Zeile angibt, wie häufig auf ein Zeichen jedes andere folgt (Abb. 6).

ABB. 6 Übergangsmatrix eines Bigramm-Modells auf Zeichenebene. Zeilen: Kontext, Spalten: nächstes Zeichen; je dunkler, desto häufiger.

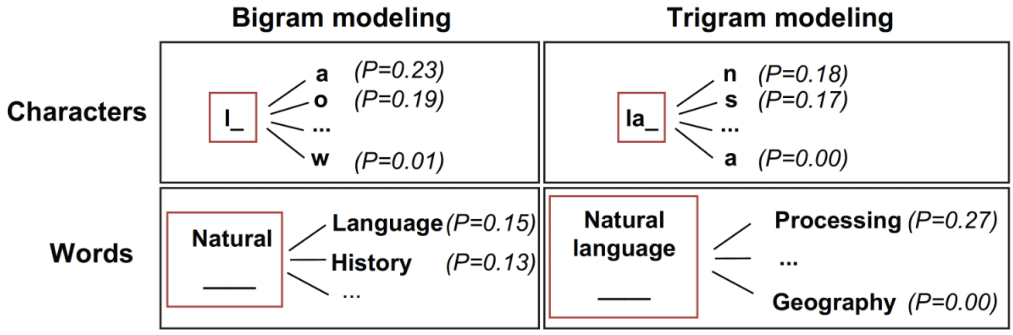


Sprachgenerierung

Das Erzeugen des nächsten Tokens ist ein Zufallsexperiment. Anders als bei einem fairen Würfel (Gleichverteilung) ziehen wir aber aus einer *gewichteten Verteilung*: Das Modell »würfelt« das nächste Zeichen, wobei Zeichen mit hoher relativer Häufigkeit eine entsprechend größere Treffer-

fläche auf diesem gedachten »Sprach-Würfel« haben (Abb. 7). Dasselbe Prinzip funktioniert auf Zeichen- wie auf Wortebene (Kochmar, 2022).

ABB. 7 Bigramm- und Trigramm-Modelle auf Zeichen- und Wortebene: Für einen Kontext liefert das Modell eine Verteilung über mögliche Folgetoken.



2.3 Skalierbarkeit und der Curse of Dimensionality

DEF 2.5 **Curse of Dimensionality**

Der **Curse of Dimensionality** beschreibt exponentielles Wachstum. Bei einem Vokabular der Größe $|V|$ gibt es für ein N-Gramm-Modell genau $|V|^N$ mögliche Kombinationen (Ziehen mit Zurücklegen unter Beachtung der Reihenfolge).

BSP 2.6 **Größe der Übergangsmatrix**

Für ein einfaches Bigramm auf Zeichenebene, ohne Sonderzeichen und ohne Unterscheidung von Groß- und Kleinschreibung, hat die Matrix $|V|^N = 26^2 = 676$ Einträge. Ein modernes Sprachmodell wie GPT-4 hat ein Vokabular von etwa 200.000 Token. Schon ein Bigramm-Modell bräuchte damit eine Matrix mit

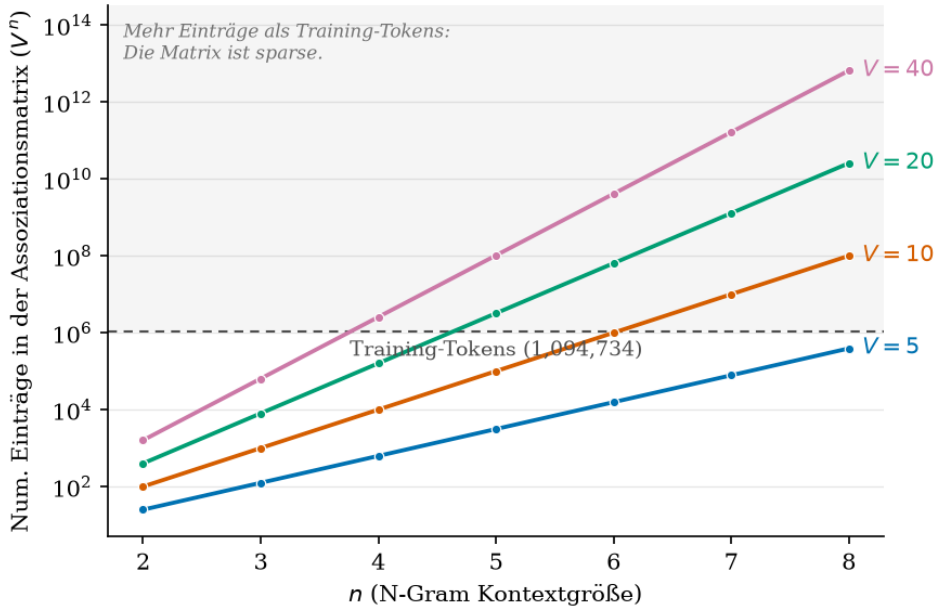
$$|V|^N = 200.000^2 = 4 \times 10^{10}$$

Einträgen. Fast alle davon wären null: Die Matrizen sind *dünn besetzt*.

Abb. 8 zeigt, wie schnell die Zahl der Einträge mit der Kontextgröße wächst. Spätestens ab der gestrichelten Linie gibt es mehr mögliche Einträge als Token in den Trainingsdaten, sodass viele *theoretisch* mögliche Folgen in den Daten nie vorkommen. Diese Kombinationen haben eine Übergangswahrscheinlichkeit von 0, und wegen der multiplikativen Verknüpfung in Gl. (2.1) ist dann die Wahrscheinlichkeit der gesamten Folge 0.

Durch *Laplace-Smoothing* wird zu jeder möglichen Zelle ein kleiner konstanter Wert addiert, sodass keine Wahrscheinlichkeit ganz verschwindet. Für das Bigramm-Modell auf Zeichenebene:

ABB. 8 Anzahl der Einträge in der Übergangsmatrix in Abhängigkeit von der Kontextgröße n, für verschiedene Vokabulargrößen. Oberhalb der gestrichelten Linie gibt es mehr Einträge als Trainings-Token.



$$P(r \mid q) = \frac{\text{Anzahl}(q \to r) + 1}{\text{Zeilensumme}(q) + 26} \tag{2.3}$$

2.4 Gedächtnisverlust

Ein N-Gramm-Modell hat ein Kontextfenster (ein »Gedächtnis«) von $N - 1$ Token. Eine Frage wie

> »John und Mary sitzen am Tisch. Mit wem sitzt John am Tisch? …«

würden Sie vermutlich problemlos mit »Mary« beantworten. Ein N-Gramm-Modell geringer Ordnung hat an dieser Stelle aber längst »vergessen«, dass es Mary jemals gab. N-Gramm-Sprachmodelle können Informationen über längere Kontexte also nicht erhalten.

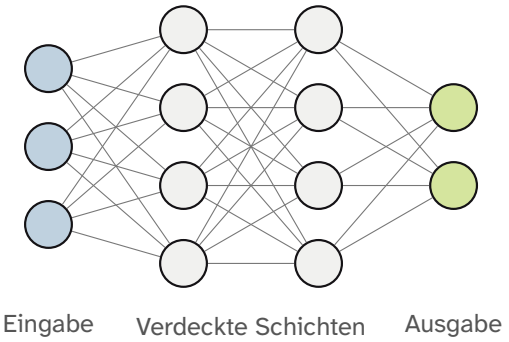
§3

Vektorrepräsentationen

3.1 Motivation

Neuronale Netze verwenden dieselben Parameter, unabhängig von der Eingabe (Abb. 9). Die Wahrscheinlichkeit einer Folge wird also durch die kombinierte Aktivierung vieler Parameter repräsentiert, statt durch eine eigene Zelle in einer riesigen Matrix. Das löst unser Skalierungsproblem zumindest teilweise.

ABB. 9 Ein vollständig verbundenes neuronales Netz: Jede Eingabe durchläuft dieselben Gewichte.



Neuronale Netze brauchen allerdings reellwertige Eingaben. Die *diskreten* Token müssen deshalb zuerst *vektorisiert* werden.

3.2 One-Hot-Vektoren

Die einfachste Idee ist ein Vektor der Größe $|V|$, in dem jedes Element einem möglichen Token entspricht. Um ein Token darzustellen, wird das zugehörige Element auf 1 gesetzt, alle anderen bleiben 0. Diese *One-Hot*-Darstellung ist simpel, hat aber drei Nachteile. Sie ist hochdimensional (bei GPT-4 wäre $|V| = 200.000$), sie ist per Definition dünn besetzt und damit ineffizient, und sie trägt keinerlei Semantik: Alle Token sind gleich weit voneinander entfernt.

3.3 Abstandsmaße

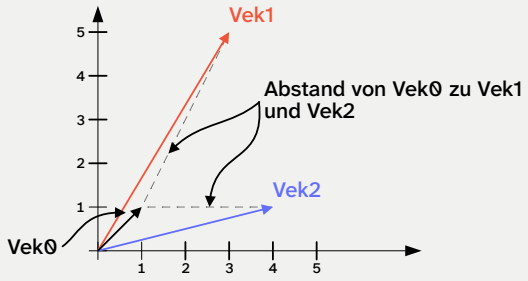
Um zu beschreiben, ob zwei Vektoren »ähnlich« sind, brauchen wir ein Abstands- oder Ähnlichkeitsmaß. Zwei Maße sind besonders verbreitet.

EXKURS **Euklidische Distanz**

$$d(p, q) = \sqrt{\sum_{i=1}^{n} (p_i - q_i)^2}$$

Für $n = 2$ entspricht die euklidische Distanz dem Satz des Pythagoras (Abb. 10). In höheren Dimensionen ist sie wenig aussagekräftig, weil sich die Abstände zufälliger Punkte immer mehr angleichen.

ABB. 10 Euklidische Distanz zwischen zwei Vektoren.

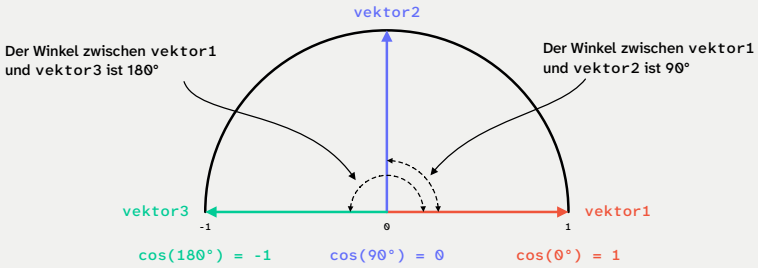


EXKURS **Kosinusähnlichkeit**

$$S_C(p, q) = \frac{p \cdot q}{\|p\| \, \|q\|}$$

Die Kosinusähnlichkeit entspricht dem Kosinus des Winkels zwischen zwei Vektoren (Abb. 11). Sie hängt nicht von der Länge der Vektoren ab und bleibt auch in hohen Dimensionen robust.

ABB. 11 Kosinusähnlichkeit: gleiche Richtung ergibt 1, orthogonale Vektoren 0, entgegengesetzte −1.



3.4 Word2Vec

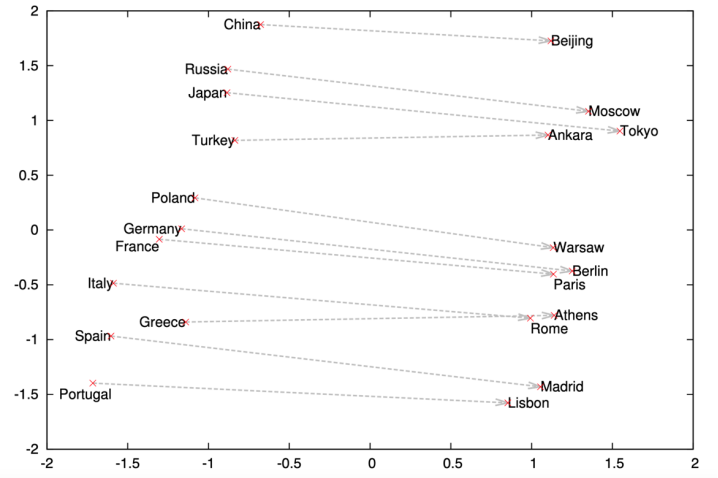
Word2Vec (Mikolov et al., 2013) beruht auf einer einfachen Beobachtung: Wörter mit ähnlicher Bedeutung tauchen in ähnlichen Kontexten auf. Jedem Wort wird ein Vektor in $\mathbb{R}^N$ zugewiesen, für ein *relativ* großes $N \in [500, 2000]$. Diese Zuweisung (die *Projektion*) wird so optimiert, dass $d(w_1, w_2) \sim d(c_1, c_2)$ gilt, wobei c_i der lexikalische Kontext von w_i ist. Liegen die Kontexte zweier Wörter im Vektorraum nah beieinander, sollen also auch die Wörter selbst nah beieinander liegen.

So lernt das Modell implizit Beziehungen zwischen Wörtern. Eine bekannte Folge ist, dass sich mit den Vektoren rechnen lässt:

$$\text{vec}(\textit{Berlin}) - \text{vec}(\textit{Germany}) + \text{vec}(\textit{France}) \approx \; ? \tag{3.1}$$

Abb. 12 zeigt, warum das funktioniert: Der Vektor vom Land zur Hauptstadt ist für viele Länder nahezu derselbe.

ABB. 12 Projektion von Word2Vec-Vektoren für Länder und ihre Hauptstädte (Mikolov et al., 2013).



3.5 Anwendungen von Embeddings

Word-Embeddings lassen sich zum Beispiel für *Information Retrieval* verwenden, ohne dass dafür ein Sprachmodell nötig ist. Das geschieht in praktisch jeder digitalen Schlagwortsuche, aber auch in RAG-Systemen, die Sprachmodelle mit dem passenden Kontext versorgen. Außerdem erleichtern Embeddings das Training des *eigentlichen* Sprachmodells: Das Trainingsziel wird einfacher und bestehende Modelle lassen sich wiederverwenden. In großen Modellen werden die Embeddings schließlich oft einfach mitgelernt.

§4

Neuronale Sprachmodelle

4.1 Definition

Ein neuronales Sprachmodell ersetzt die gezählten Wahrscheinlichkeiten aus Def 2.1 durch ein neuronales Netz:

DEF 4.1 **Neuronales Sprachmodell**

f sei ein neuronales Netz mit Parametern θ und w_1^T eine Wortfolge der Länge T. Dann ist

$$\hat{P}(w_T \mid w_1^{T-1}) = f(w_1^{T-1}, \theta) \tag{4.1}$$

ein **neuronales Sprachmodell**.

4.2 Attention

Frühe neuronale Sprachmodelle konnten Informationen zwar über längere Folgen verarbeiten als N-Gramm-Modelle, litten aber unter einem allmählichen Verlust von Kontextinformation. Solche *rekurrenten neuronalen Netze* geben Informationen aus vergangenen Durchläufen *direkt verkettet* an spätere Durchläufe weiter. Damit ist es besonders schwierig, Informationen nach ihrer *Relevanz* statt nach ihrer Position zu verarbeiten und über längere Folgen hinweg zu priorisieren.

Attention (Vaswani et al., 2017) löst dieses Problem. Aus jedem Token in einer Folge von Token-Embeddings werden mit gelernten Projektionen ein *Query*-, ein *Key*- und ein *Value*-Vektor berechnet, gesammelt in den Matrizen Q, K und V:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{4.2}$$

Die Softmax-Verteilung sorgt dafür, dass je nach Höhe des Skalarprodukts zwischen *Query* und *Key* ein entsprechender Anteil des zum *Key* gehörenden *Value* in das Ergebnis einfließt. Der Faktor $1/\sqrt{d_k}$ hält die Verteilung auch in hohen Dimensionen stabil; die Herleitung steht im Anhang. Nach einer Attention-Schicht enthält die Folge also nicht nur Informationen zur Bedeutung der einzelnen Token (aus den Embeddings), sondern auch zu ihrem Kontext (Abb. 13).

4.3 Transformer

Abb. 14 zeigt einen *Decoder-only-Transformer*, die Standardarchitektur moderner großer Sprachmodelle. Transformer nutzen keine Rekurrenz, geben also keine Informationen aus vorherigen Durchläufen weiter, und die reinen Matrixmultiplikationen in ihren Schichten sind invariant gegenüber der Reihenfolge der Token. Die Token-Embeddings brauchen deshalb eine additive Komponente, aus der sich die Position jedes Tokens ablesen lässt (»+ Position«).

Am Ende wird eine Softmax-Verteilung über das Vokabular aufgespannt. Die Auswahl des nächsten Tokens ist also effektiv eine *Klassifikation*.

4.4 Autoregression

ABB. 13 Attention in einem Transformer: Aus jedem Token entstehen Query, Key und Value; die Gewichte der Attention verteilen den Kontext. Visualisierung mit dem Transformer Explainer.

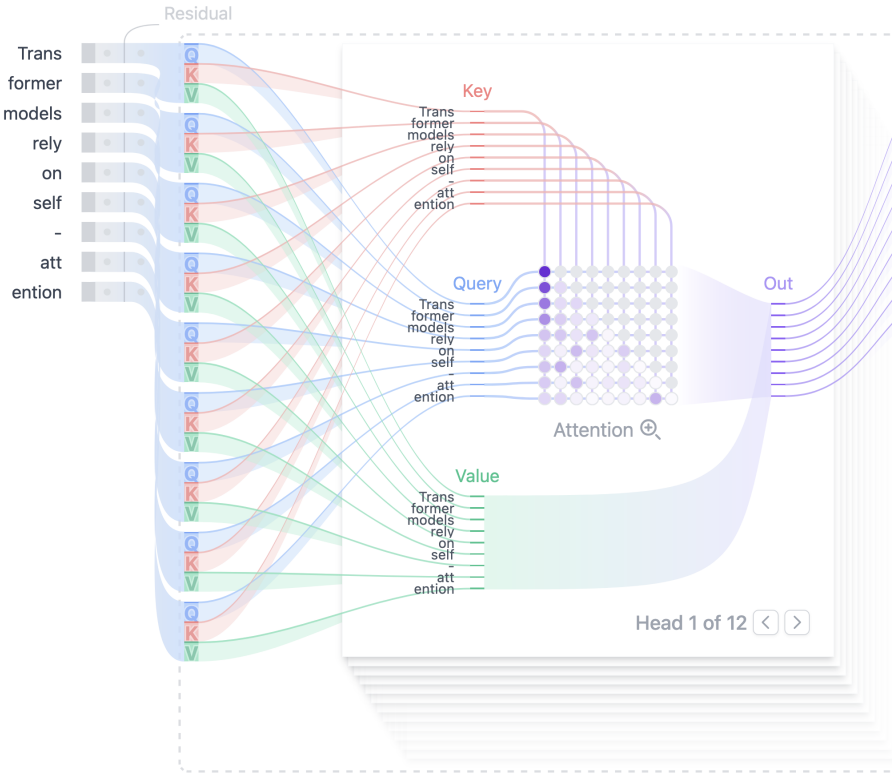
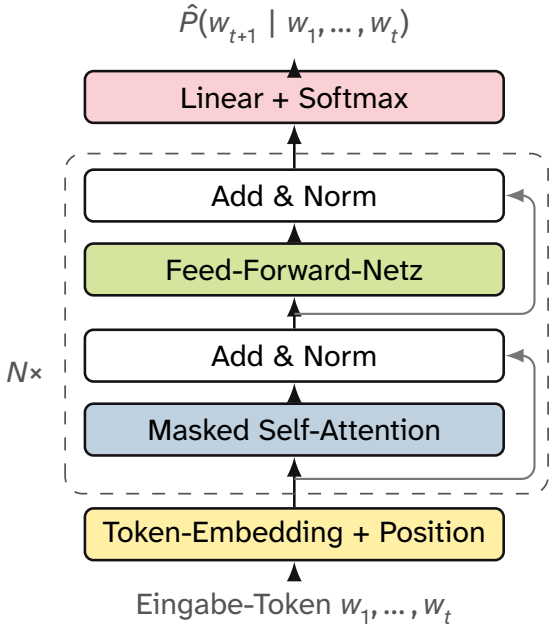


ABB. 14 Aufbau eines Decoder-only-Transformers. Der gestrichelte Block wird *N*-mal hintereinander ausgeführt; graue Pfeile sind Residualverbindungen.



WICHTIG **Sprachgenerierung ist Klassifikation**

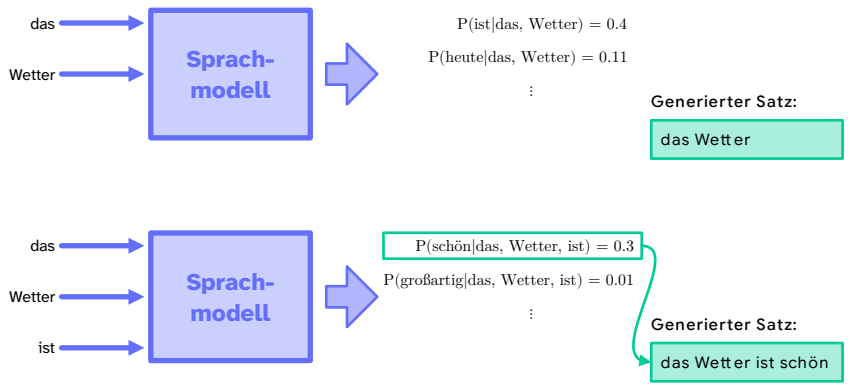
Das Modell gibt eine Wahrscheinlichkeitsverteilung über das Vokabular aus: Wie wahrscheinlich ist es, dass *dieses* Token das nächste in der Folge ist? Das nächste Token wird entweder deterministisch gewählt, indem man das wahrscheinlichste nimmt, oder es wird gewichtet aus der Verteilung »gezogen«.

DEF 4.2 **Autoregression**

Transformer erzeugen Text Schritt für Schritt. In jedem Schritt wird das erzeugte Token an die Kontextfolge angehängt und beeinflusst so die Erzeugung des nächsten Tokens; das nennt man **Autoregression**. Bei der Berechnung der *Attention* kann sich jede Position nur auf vorherige Positionen der Folge beziehen (*kausale Maske*).

Abb. 15 zeigt zwei Schritte dieses Ablaufs: Das Modell erhält »das Wetter«, zieht aus seiner Ausgabeverteilung das nächste Wort, hängt es an die Bedingung an und wiederholt den Vorgang, bis der Satz »das Wetter ist schön« entstanden ist.

ABB. 15 Sprachgenerierung als wiederholtes Ziehen: oben der erste Schritt, unten der letzte.



Das Notebook zu großen Sprachmodellen aus dem Workshop ist unter tinyurl.com/smw-llms-nb zu finden (Abb. 16).

ABB. 16 Notebook zu großen Sprachmodellen: tinyurl.com/smw-llms-nb.

4.5 Vom Sprachmodell zum Dialogsystem

Was braucht ein Sprachmodell, damit es zum Chat-Modell wird? Drei Eigenschaften sind entscheidend.

- **Antworten auf Fragen.** Statt frei zu assoziieren, also einen Text einfach fortzusetzen, soll das Modell tatsächlich auf Fragen antworten.
- **Langer Kontext.** Das Modell soll lange Gesprächsverläufe als Kontext für seine Antworten heranziehen können. Nutzer:innen sollten es nicht alle drei Nachrichten an wichtige Informationen »erinnern« müssen.
- **Geschwindigkeit.** Das Modell soll möglichst schnell antworten. Da Grafikprozessoren (GPUs) eigens für Matrixmultiplikationen gebaut sind, skalieren neuronale Sprachmodelle gut mit Rechenleistung.

Erst mit Transformern wurden Sprachmodelle so mächtig, dass Anwendungen wie ChatGPT möglich wurden.

§5

Zusammenfassung

Tab. 1 fasst den Weg dieses Skripts zusammen: Jede Station löst ein Problem der vorherigen und hat selbst eine Grenze, die zur nächsten führt.

TAB. 1 Die vier Stationen von der Regel zum Transformer.

Station	Kernpunkte	Grenze
§1 Grundlagen	Regelbasiert (ELIZA), dann statistisch; maschinelles Lernen: Leistung steigt mit Erfahrung; neuronale Netze als differenzierbare Funktionen mit lernbaren Gewichten	Regeln skalieren nicht auf die Vielfalt natürlicher Sprache
§2 N-Gramme	Sprache als Wahrscheinlichkeit $\hat{P}(w_i \mid w_1, \ldots, w_{i-1})$; Markov-Annahme; Training durch Zählen, Generierung durch »Würfeln«	$\lvert V\rvert^N$ wächst exponentiell, Gedächtnis nur $N-1$ Token
§3 Embeddings	Token werden reellwertige Vektoren; One-Hot ist simpel, aber riesig und ohne Semantik; Word2Vec: ähnliche Bedeutung, ähnliche Vektoren	Ein Wortvektor allein kennt seinen Kontext nicht

Station	Kernpunkte	Grenze
§4 Transformer	Attention gewichtet Kontext nach *Relevanz* statt Position; parallel auf GPUs; Autoregression: nächstes Token als Klassifikation	Grundlage für Chat-Modelle wie ChatGPT

WICHTIG **Kernidee**

Ein Sprachmodell sagt immer nur das *nächste Token* vorher.

Literatur

Bengio, Y., Ducharme, R., Vincent, P., & Jauvin, C. (2003). A Neural Probabilistic Language Model. *Journal of Machine Learning Research*, *3*, 1137–1155.

Kochmar, E. (2022). *Getting Started with Natural Language Processing*. Simon and Schuster.

Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013). Distributed Representations of Words and Phrases and Their Compositionality. *Advances in Neural Information Processing Systems*, *26*.

Mitchell, T. M. (1997). *Machine Learning*. McGraw Hill.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*, *30*.

Veritasium. (2025). *The Strange Math That Predicts (Almost) Anything*. YouTube. https://www.youtube.com/watch?v=KZeIEiBrT_w

Weizenbaum, J. (1966). ELIZA—a Computer Program for the Study of Natural Language Communication between Man and Machine. *Communications of the ACM*, *9*(1), 36–45. https://doi.org/10.1145/365153.365168

Anhang: Herleitung des Skalierungsfaktors

Warum teilt die skalierte Dot-Product-Attention in Gl. (4.2) durch $\sqrt{d_k}$? Wir nehmen an, dass die Komponenten von Q und K unabhängig sind, mit Erwartungswert $\mu = 0$ und Varianz $\sigma^2 = 1$. Für einen Eintrag $S_{ij} = \sum_l Q_{il} K_{jl}$ der Score-Matrix gilt dann $\mathbb{E}[S_{ij}] = 0$ und

$$
\begin{aligned}
\mathrm{Var}(S_{ij}) &= \mathbb{E}\Big[\Big(\textstyle\sum_l Q_{il}K_{jl}\Big)^2\Big] && (\text{da } \mathbb{E}[S_{ij}] = 0)\\
&= \textstyle\sum_l \mathbb{E}[Q_{il}^2]\,\mathbb{E}[K_{jl}^2] + \sum_{l\neq r} \underbrace{\mathbb{E}[Q_{il}]\,\mathbb{E}[K_{jl}]\,\mathbb{E}[Q_{ir}]\,\mathbb{E}[K_{jr}]}_{=0} && (\text{Unabhängigkeit})\\
&= \textstyle\sum_{l=1}^{d_k} 1 \cdot 1 = d_k.
\end{aligned}
$$

Die Scores haben also die Varianz d_k. Teilt man sie durch $\sqrt{d_k}$, ist die Varianz wieder 1, und die Softmax sättigt auch bei großen d_k nicht.

Über die Autoren

Erik Voigt (erik.voigt@uni-bamberg.de) und Johannes Langer (johannes.langer@uni-bamberg.de) sind wissenschaftliche Mitarbeiter am Lehrstuhl für Kognitive Systeme der Universität Bamberg und forschen und lehren im Bereich Künstliche Intelligenz und maschinelles Lernen.